دانشگاه صنعتی شریف
دانشکده‌ی مهندسی کامپیوتر

یادگیری ماشین

پاییز و زمستان ۱۴۰۴

استاد: علی شریفی زارچی

گردآورندگان:

تمرین چهارم | موضوع | مهلت ارسال: ۱۷ بهمن

سوالات نظری (۱۰۰ نمره)

۱. (۲۰ نمره) جمله زیر را در نظر بگیرید:

$$S = [w_1, \dots, w_7] = [\text{Cats, chase, mice, in, the, dark, night}]$$

و یک پنجره متقارن با اندازه $m = 2$ (یعنی کلمات زمینه در صورت وجود، در موقعیت‌های $\pm 1, \pm 2$ قرار دارند). در این سوال از پارامتردهی استاندارد Word2Vec با دو جدول embedding استفاده می‌کنیم: بردارهای مرکز/ورودی به شکل $v_w \in \mathbb{R}^d$ و بردارهای زمینه/خروجی به شکل $u_w \in \mathbb{R}^d$ نمایش داده می‌شوند.

(آ)
- تمام جفت‌های آموزشی مثبت (w_c, w_o) استخراج‌شده از جمله در روش Skip-gram را فهرست کنید.
- نمونه‌های آموزشی CBOW به‌صورت $(\{\text{context}\} \to w_c)$ را فقط برای مراکزی که زمینه کامل دارند (موقعیت‌های ۳, ۴, ۵) فهرست کنید.

استخراج جفت‌های آموزشی مثبت در Skip-gram: در روش Skip-gram هر کلمه‌ی مرکزی w_c سعی می‌کند کلمات زمینه‌ی w_o اطراف خود را پیش‌بینی کند. بنابراین، تمام جفت‌های مثبت (w_c, w_o) از روی پنجره‌ی زمینه استخراج می‌شوند. همچنین می‌دانیم در Skip-gram ترتیب اهمیت ندارد و هر زوج (w_c, w_o) یک نمونه‌ی آموزشی مستقل محسوب می‌شود.

- مرکز: **Cats**

$$(\text{Cats, chase}),\ (\text{Cats, mice})$$

- مرکز: **chase**

$$(\text{chase, Cats}),\ (\text{chase, mice}),\ (\text{chase, in})$$

- مرکز: **mice**

$$(\text{mice, Cats}),\ (\text{mice, chase}),\ (\text{mice, in}),\ (\text{mice, the})$$

- مرکز: **in**

$$(\text{in, chase}),\ (\text{in, mice}),\ (\text{in, the}),\ (\text{in, dark})$$

- مرکز: **the**

$$(\text{the, mice}),\ (\text{the, in}),\ (\text{the, dark}),\ (\text{the, night})$$

- مرکز: **dark**

$$(\text{dark, in}),\ (\text{dark, the}),\ (\text{dark, night})$$

- مرکز: **night**

$$(\text{night, the}),\ (\text{night, dark})$$

نمونه‌های آموزشی CBOW برای مراکزی با زمینه‌ی کامل: در روش CBOW مجموعه‌ی کلمات زمینه به‌عنوان ورودی داده می‌شود و کلمه‌ی مرکزی پیش‌بینی می‌گردد:

$$\{\text{context}\} \to w_c$$

طبق صورت سؤال، فقط کلماتی را در نظر می‌گیریم که زمینه‌ی کامل در پنجره‌ی ± ۲ دارند، یعنی موقعیت‌های ۳، ۴ و ۵ جمله.

- مرکز: **mice** (موقعیت ۳)

$$\{\text{Cats}, \text{chase}, \text{in}, \text{the}\} \to \text{mice}$$

- مرکز: **in** (موقعیت ۴)

$$\{\text{chase}, \text{mice}, \text{the}, \text{dark}\} \to \text{in}$$

- مرکز: **the** (موقعیت ۵)

$$\{\text{mice}, \text{in}, \text{dark}, \text{night}\} \to \text{the}$$

در CBOW بردارهای زمینه (معمولاً با جمع یا میانگین گرفتن از v_wها) ترکیب می‌شوند تا کلمه‌ی مرکزی پیش‌بینی شود، بر خلاف Skip-gram که از یک کلمه‌ی مرکزی برای پیش‌بینی تک‌تک کلمات زمینه استفاده می‌کند.

(ب) یک مشاهده تجربی رایج این است که Skip-gram معمولا برای واژه‌های کم‌بسامد نمایش‌های قوی‌تری نسبت به CBOW یاد می‌گیرد. با استفاده از اندازه پنجره m و واژه w با بسامد $f(w)$، یک استدلال کوتاه و مبتنی بر ریاضیات ارائه دهید که نشان دهد Skip-gram سیگنال یادگیری قوی‌تری برای واژه‌های کم‌بسامد نسبت به CBOW اعمال می‌کند.

در Skip-gram هر بار که واژه‌ی w به‌عنوان کلمه‌ی مرکزی ظاهر می‌شود، تا $۲m$ نمونه‌ی آموزشی مستقل تولید می‌گردد:

$$(w_c = w,\ w_o \in \text{context}(w))$$

بنابراین، تعداد کل به‌روزرسانی‌هایی که مستقیماً بردار مرکزی v_w را تحت تأثیر قرار می‌دهند، متناسب است با:

$$\mathcal{O}\big(f(w) \times ۲m\big)$$

نکته‌ی کلیدی این است که در هر نمونه، گرادیان مستقیماً از پیش‌بینی یک کلمه‌ی زمینه ناشی می‌شود و سیگنال یادگیری متمرکز و کم‌نویز است.

در CBOW واژه‌ی w تنها زمانی به‌عنوان خروجی (هدف) ظاهر می‌شود که در مرکز پنجره قرار گیرد:

$$\{\text{context}\} \to w$$

در این حالت، بردار v_w از طریق میانگین (یا جمع) بردارهای زمینه به‌روزرسانی می‌شود. تعداد این به‌روزرسانی‌ها از مرتبه‌ی:

$$\mathcal{O}\big(f(w)\big)$$

است و علاوه بر آن، سیگنال گرادیان بین چندین واژه‌ی زمینه تقسیم می‌شود که باعث تضعیف سیگنال یادگیری برای هر واژه‌ی منفرد می‌گردد.

لذا از آن‌جا که Skip-gram به‌روزرسانی‌های بیشتری برای هر وقوع واژه ایجاد می‌کند و گرادیان‌های متمرکزتری ارائه می‌دهد، برای واژه‌های کم‌بسامد سیگنال یادگیری قوی‌تری نسبت به CBOW اعمال می‌کند و در نتیجه نمایش‌های معنایی بهتری برای آن‌ها می‌آموزد.

۲. (۲۰ نمره) Multi-Head Attention (MHA) را برای ورودی $X \in \mathbb{R}^{n\times d}$ به‌صورت زیر تعریف کنید:

$$\text{head}_r = \text{Att}\left(XW_Q^{(r)},\, XW_K^{(r)},\, XW_V^{(r)}\right),$$
$$\text{MHA}(X) = \text{Concat}(\text{head}_۱, \ldots, \text{head}_h)\, W_O$$

که در آن داریم:

$$\text{Att}(Q,K,V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

(آ) توضیح دهید کاهش دادن MHA به یک لایه تک‌سری با پارامترهای $(\bar{W}_Q, \bar{W}_K, \bar{W}_V, \bar{W}_O)$ به چه معناست (به‌صورت برابری دو تابع برای همه ورودی‌های معتبر).

طبق تعریف سوال، خروجی چند سر حاصل Concat خروجی تمام head ها ضرب در ماتریس W_O است. ما می‌توانیم ماتریس W_O را به h زیرـماتریس (بلوک سطری) تقسیم کنیم که هر $W_O^{(r)}$ مربوط به ابعاد خروجی $head_r$ باشد. بنابراین، عملیات Concat و ضرب ماتریسی را می‌توان به صورت مجموع زیر نوشت:

$$\text{MHA}(X) = \sum_{r=۱}^{h} \text{head}_r W_O^{(r)}$$

با جایگذاری تعریف دقیق هر head، خواهیم داشت:

$$\text{MHA}(X) = \sum_{r=۱}^{h} \left[\text{softmax}\left(\frac{XW_Q^{(r)}(W_K^{(r)})^\top X^\top}{\sqrt{d_k}}\right) XW_V^{(r)}\right] W_O^{(r)}$$

در حالت Single-Head ما تنها یک ماتریس Attention-Score (خروجی سافت‌مکس) داریم. در حالی که در MHA هر سر می‌تواند الگوی توجه متفاوتی داشته باشد. برای اینکه مجموع بالا به یک فرم تک‌سری تبدیل شود، الگوی توجه تمام سرها باید یکسان باشد. یعنی ماتریس‌های Query و Key در تمام سرها باید به گونه‌ای باشند که حاصل‌ضرب آن‌ها یکسان شود:

$$\forall i,j \in \{۱,\ldots,h\}: \quad W_Q^{(i)}(W_K^{(i)})^\top = W_Q^{(j)}(W_K^{(j)})^\top$$

اگر این شرط برقرار باشد، می‌توانیم ماتریس توجه مشترک را A بنامیم و آن را فاکتور بگیریم:

$$A = \text{softmax}\left(\frac{XW_Q^{(۱)}(W_K^{(۱)})^\top X^\top}{\sqrt{d_k}}\right)$$

با فاکتور گرفتن A و ورودی X، معادله MHA ساده می‌شود:

$$\text{MHA}(X) = AX\left(\sum_{r=۱}^{h} W_V^{(r)} W_O^{(r)}\right)$$

حال این رابطه را با فرمول استاندارد یک لایه تک‌سری مقایسه می‌کنیم:

$$\text{SHA}(X) = \text{softmax}\left(\frac{X\bar{W}_Q\bar{W}_K^\top X^\top}{\sqrt{d_k}}\right) X\bar{W}_V\bar{W}_O$$

با تطبیق دو رابطه، پارامترهای لایه معادل به صورت زیر تعریف می‌شوند:

- **پارامترهای توجه ($\bar{W}_Q, \bar{W}_K$):** کافی است برابر با پارامترهای یکی از سرها (مثلاً سر اول) باشند، زیرا فرض کردیم همه یکسان عمل می‌کنند:

$$\bar{W}_Q = W_Q^{(۱)}, \quad \bar{W}_K = W_K^{(۱)}$$

- **پارامترهای خروجی ($\bar{W}_V, \bar{W}_O$):** ماتریس مقدار (Value) جدید، مجموع ترکیب‌شده‌ی تمام مقدارهای قبلی است. برای سادگی می‌توانیم $\bar{W}_O$ را ماتریس همانی (I) در نظر بگیریم و تمام وزن‌ها را در $\bar{W}_V$ تجمیع کنیم:

$$\bar{W}_V = \sum_{r=۱}^{h} W_V^{(r)} W_O^{(r)}, \quad \bar{W}_O = I$$

کاهش دادن MHA به یک لایه تک‌سری به این معناست که تمام سرها به مکان‌های یکسانی توجه می‌کنند. در حالی که فلسفه‌ی وجودی Multi-Head این است که مدل بتواند همزمان به جنبه‌های مختلف متن (مثلاً روابط نحوی در یک سر و روابط معنایی در سر دیگر) توجه کند. اگر این لایه قابل کاهش باشد، یعنی مدل از ظرفیت خود استفاده نکرده و عملاً تمام سرها دارند یک کار تکراری را روی فضاهای برداری مختلف انجام می‌دهند.

(ب) یک حالت مشخص (با n, h, d_k, d_v کوچک) و انتخاب‌های صریح از پارامترها بسازید که در آن $\mathrm{MHA}(X)$ خروجی‌ای تولید کند که برای هیچ انتخابی از پارامترهای تک‌سری نتوان آن را به‌شکل

$$\mathrm{Att}(X\bar{W}_Q, X\bar{W}_K, X\bar{W}_V)\bar{W}_O$$

نوشت. عدم امکان را اثبات کنید.

برای اثبات اینکه MHA قدرتمندتر از Single-Head است، از استدلال **رتبه ماتریس (Rank)** استفاده می‌کنیم. نشان می‌دهیم که MHA می‌تواند خروجی‌هایی با رتبه بالاتر تولید کند که SHA با d_v محدود قادر به تولید آن نیست. مقادیر زیر را در نظر می‌گیریم:

$$n = ۲, \quad h = ۲, \quad d_k = ۱, \quad d_v = ۱$$

ورودی X را ماتریس همانی فرض می‌کنیم: $X = \begin{bmatrix} ۱ & ۰ \\ ۰ & ۱ \end{bmatrix}$. فرض می‌کنیم وزن‌های دو سر به گونه‌ای تنظیم شده‌اند که:

- **Head ۱:** بردار ویژگی اول را استخراج کند: $\mathrm{head}_۱ = \begin{bmatrix} ۱ \\ ۰ \end{bmatrix}$
- **Head ۲:** بردار ویژگی دوم را استخراج کند: $\mathrm{head}_۲ = \begin{bmatrix} ۰ \\ ۱ \end{bmatrix}$

خروجی نهایی (با فرض Concat و $W_O = I$) برابر است با:

$$Y_{MHA} = [\mathrm{head}_۱, \mathrm{head}_۲] = \begin{bmatrix} ۱ & ۰ \\ ۰ & ۱ \end{bmatrix}$$

رتبه این ماتریس برابر است با: $\mathrm{rank}(Y_{MHA}) = ۲$. در لایه تک‌سری با $d_v = ۱$، خروجی به صورت زیر محاسبه می‌شود:

$$Y_{SHA} = \underbrace{A}_{\text{Attention}} \underbrace{(X\bar{W}_V)}_{۲\times۱} \bar{W}_O$$

ماتریس میانی $(X\bar{W}_V)$ دارای ابعاد ۲×۱ است، بنابراین رتبه آن حداکثر ۱ است. در جبر خطی می‌دانیم که ضرب ماتریس‌ها نمی‌تواند رتبه را افزایش دهد:

$$\mathrm{rank}(Y_{SHA}) \leq \mathrm{rank}(X\bar{W}_V) \leq d_v = ۱$$

چون $\text{rank}(Y_{MHA}) = ۲$ اما $\text{rank}(Y_{SHA}) \leq ۱$، امکان ندارد که لایه تک‌سری بتواند خروجی لایه چندسری را در این مثال تولید کند.

۳. (۲۰ نمره) فرض کنید یک مدل انکودر Transformer مانند BERT دارید که فقط شامل self-attention است و feed-forward network ندارد. ورودی شما جمله‌ای با سه توکن است: "I love AI" که پس از tokenization به توکن‌های

$$[\text{CLS}],\ I,\ \text{love},\ \text{AI},\ [\text{SEP}]$$

تبدیل می‌شود. فرض کنید embedding اولیه هر توکن یک بردار دوبعدی است:

$$[\text{CLS}] = [۱, ۰], \quad I = [۰, ۱], \quad \text{love} = [۱, ۱], \quad \text{AI} = [۰, ۰], \quad [\text{SEP}] = [-۱, ۰]$$

ماتریس‌های Query (Q)، Key (K) و Value (V) به‌صورت زیر تعریف شده‌اند:

$$W_Q = \begin{pmatrix} ۱ & ۰ \\ ۰ & ۱ \end{pmatrix}, \quad W_K = \begin{pmatrix} ۱ & ۱ \\ ۰ & ۱ \end{pmatrix}, \quad W_V = \begin{pmatrix} ۱ & ۰ \\ ۱ & ۱ \end{pmatrix}$$

(آ) بردارهای Q، K و V را برای هر توکن محاسبه کنید.

ما ۵ توکن ورودی داریم که هر کدام یک بردار تعبیه (Embedding) دو بعدی هستند. ماتریس ورودی X را به صورت زیر تشکیل می‌دهیم:

$$X = \begin{pmatrix} ۱ & ۰ \\ ۰ & ۱ \\ ۱ & ۱ \\ ۰ & ۰ \\ -۱ & ۰ \end{pmatrix} \begin{array}{l} \leftarrow [\text{CLS}] \\ \leftarrow \text{I} \\ \leftarrow \text{love} \\ \leftarrow \text{AI} \\ \leftarrow [\text{SEP}] \end{array}$$

ماتریس‌های وزن W_Q, W_K, W_V نیز داده شده‌اند. برای محاسبه بردارها از ضرب ماتریسی استفاده می‌کنیم. فرمول محاسبه $Q = XW_Q$ است. از آنجا که W_Q ماتریس همانی است، بردارها تغییری نمی‌کنند:

$$Q = X \begin{pmatrix} ۱ & ۰ \\ ۰ & ۱ \end{pmatrix} = \begin{pmatrix} ۱ & ۰ \\ ۰ & ۱ \\ ۱ & ۱ \\ ۰ & ۰ \\ -۱ & ۰ \end{pmatrix}$$

فرمول محاسبه $K = XW_K$ است. ضرب ماتریس ورودی در $W_K = \begin{pmatrix} ۱ & ۱ \\ ۰ & ۱ \end{pmatrix}$ به صورت زیر انجام می‌شود:

$$K = \begin{pmatrix} ۱ & ۰ \\ ۰ & ۱ \\ ۱ & ۱ \\ ۰ & ۰ \\ -۱ & ۰ \end{pmatrix} \times \begin{pmatrix} ۱ & ۱ \\ ۰ & ۱ \end{pmatrix} = \begin{pmatrix} ۱ & ۱ \\ ۰ & ۱ \\ ۱ & ۲ \\ ۰ & ۰ \\ -۱ & -۱ \end{pmatrix}$$

فرمول محاسبه $V = XW_V$ است. ضرب ماتریس ورودی در $W_V = \begin{pmatrix} ۱ & ۰ \\ ۱ & ۱ \end{pmatrix}$ به صورت زیر است:

$$V = \begin{pmatrix} ۱ & ۰ \\ ۰ & ۱ \\ ۱ & ۱ \\ ۰ & ۰ \\ -۱ & ۰ \end{pmatrix} \times \begin{pmatrix} ۱ & ۰ \\ ۱ & ۱ \end{pmatrix} = \begin{pmatrix} ۱ & ۰ \\ ۱ & ۱ \\ ۲ & ۱ \\ ۰ & ۰ \\ -۱ & ۰ \end{pmatrix}$$

برای هر توکن، بردارهای بدست آمده عبارتند از:

- **[CLS]**: $q = [۱, ۰], \quad k = [۱, ۱], \quad v = [۱, ۰]$
- **I**: $q = [۰, ۱], \quad k = [۰, ۱], \quad v = [۱, ۱]$
- **love**: $q = [۱, ۱], \quad k = [۱, ۲], \quad v = [۲, ۱]$
- **AI**: $q = [۰, ۰], \quad k = [۰, ۰], \quad v = [۰, ۰]$
- **[SEP]**: $q = [-۱, ۰], \quad k = [-۱, -۱], \quad v = [-۱, ۰]$

(ب) attention scores را برای هر توکن نسبت به سایر توکن‌ها محاسبه کنید (با استفاده از softmax روی $\frac{QK^\top}{\sqrt{d_k}}$ که در آن $d_k = ۲$ است).

با استفاده از ماتریس‌های Q و K که در قسمت (آ) بدست آوردیم:

$$QK^\top = \begin{pmatrix} ۱ & ۰ \\ ۰ & ۱ \\ ۱ & ۱ \\ ۰ & ۰ \\ -۱ & ۰ \end{pmatrix} \times \begin{pmatrix} ۱ & ۰ & ۱ & ۰ & -۱ \\ ۱ & ۱ & ۲ & ۰ & -۱ \end{pmatrix} = \begin{pmatrix} ۱ & ۰ & ۱ & ۰ & -۱ \\ ۱ & ۱ & ۲ & ۰ & -۱ \\ ۲ & ۱ & ۳ & ۰ & -۲ \\ ۰ & ۰ & ۰ & ۰ & ۰ \\ -۱ & ۰ & -۱ & ۰ & ۱ \end{pmatrix}$$

تمام درایه‌ها را بر $\sqrt{۲} \approx ۱/۴۱$ تقسیم می‌کنیم تا ماتریس Logits Scaled بدست آید (S):

$$S = \frac{QK^\top}{\sqrt{۲}} \approx \begin{pmatrix} ۰/۷۱ & ۰ & ۰/۷۱ & ۰ & -۰/۷۱ \\ ۰/۷۱ & ۰/۷۱ & ۱/۴۱ & ۰ & -۰/۷۱ \\ ۱/۴۱ & ۰/۷۱ & ۲/۱۲ & ۰ & -۱/۴۱ \\ ۰ & ۰ & ۰ & ۰ & ۰ \\ -۰/۷۱ & ۰ & -۰/۷۱ & ۰ & ۰/۷۱ \end{pmatrix}$$

تابع Softmax را روی هر سطر اعمال می‌کنیم ($e^{s_{ij}} / \sum_k e^{s_{ik}}$). نتایج نهایی به صورت زیر است:

$$\text{Matrix Attention} \approx \begin{pmatrix} ۰/۳۱ & ۰/۱۵ & ۰/۳۱ & ۰/۱۵ & ۰/۰۸ \\ ۰/۲۱ & ۰/۲۱ & ۰/۴۳ & ۰/۱۰ & ۰/۰۵ \\ ۰/۲۶ & ۰/۱۳ & ۰/۵۳ & ۰/۰۶ & ۰/۰۲ \\ ۰/۲۰ & ۰/۲۰ & ۰/۲۰ & ۰/۲۰ & ۰/۲۰ \\ ۰/۱۰ & ۰/۲۰ & ۰/۱۰ & ۰/۲۰ & ۰/۴۰ \end{pmatrix}$$

(ج) خروجی self-attention را برای توکن [CLS] محاسبه کنید و توضیح دهید چگونه این خروجی می‌تواند به‌عنوان embedding نهایی متن استفاده شود.

برای محاسبه خروجی لایه توجه برای توکن اول (Z_{CLS})، ما حاصل‌ضرب داخلی بردارهای Value در وزن‌های توجه مربوط به سطر اول ماتریس توجه (A) را محاسبه می‌کنیم.

- بردارهای Value:

$$v_۱ = [۱, ۰], v_۲ = [۱, ۱], v_۳ = [۲, ۱], v_۴ = [۰, ۰], v_۵ = [-۱, ۰]$$

- سطر اول ماتریس توجه (که میزان توجه توکن CLS به سایر توکن‌ها را نشان می‌دهد):

$$[۰/۳۱, ۰/۱۵, ۰/۳۱, ۰/۱۵, ۰/۰۸]$$

محاسبه مجموع وزن‌دار بردارهای Value بر اساس امتیاز توجه توکن CLS:

$$Z_{\text{CLS}} = \sum_{j=۱}^{۵} \alpha_{۱,j} v_j$$

$$Z_{\text{CLS}} = ۰/۳۱ \begin{pmatrix} ۱ \\ ۰ \end{pmatrix} + ۰/۱۵ \begin{pmatrix} ۱ \\ ۱ \end{pmatrix} + ۰/۳۱ \begin{pmatrix} ۲ \\ ۱ \end{pmatrix} + ۰/۱۵ \begin{pmatrix} ۰ \\ ۰ \end{pmatrix} + ۰/۰۸ \begin{pmatrix} -۱ \\ ۰ \end{pmatrix} = [۱/۰۰, ۰/۴۶]$$

نحوه استفاده از این خروجی به عنوان Embedding نهایی متن:
در معماری ترانسفورمر، مکانیزم Self-Attention به هر توکن اجازه می‌دهد تا به تمام توکن‌های دیگر جمله «نگاه کند». توکن [CLS] چون در موقعیت اول قرار دارد و معنای لغوی خاصی ندارد، پس از عبور از لایه‌های توجه، بردار آن حاوی اطلاعات فشرده شده از کل جمله می‌شود. همچنین در مدل‌هایی مانند BERT، معمولاً بردار خروجی متناظر با توکن [CLS] جدا شده و به عنوان نماینده‌ی کل جمله به یک لایه Classifier (مثلاً برای تحلیل احساسات) داده می‌شود. برخلاف توکن‌های دیگر که نمایش‌دهنده‌ی کلمات خاص هستند، Z_{CLS} نمایش‌دهنده‌ی کل ساختار معنایی و نحوی جمله است.

(د) مزایای زمینه دوجهتی را در این embedding در مقایسه با یک مدل decoder-only مانند GPT توصیف کنید و مثالی بزنید که نشان دهد برداشت از معنای کلمه love چگونه می‌تواند متفاوت باشد.

در مدل‌های Decoder-only از مکانیزمی به نام Masked-Self-Attention استفاده می‌شود. این یعنی هر توکن در زمان پردازش، فقط اجازه دارد به توکن‌های **قبل از خود** (سمت چپ) توجه کند. این ساختار برای «تولید متن» (Text-Generation) ضروری است، زیرا مدل نباید آینده را ببیند. در مدل این سوال (Encoder) هیچ ماسکی وجود ندارد. توکن‌ها دسترسی کامل به **تمام جمله** (چپ و راست) دارند. این باعث می‌شود که بازنمایی (Embedding) هر کلمه با آگاهی کامل از کل بافت جمله شکل بگیرد.

مثال: تفاوت معنایی در کلمه "love" در جمله: "I-love-AI"

- **در مدل GPT (یک‌جهتی):** وقتی مدل به کلمه love می‌رسد، تنها چیزی که می‌بیند "I-love" است. مدل هنوز نمی‌داند مفعول جمله چیست. لذا Embedding کلمه love در اینجا یک معنای عام دارد (احساس علاقه فاعل). مدل نمی‌داند این علاقه به یک شخص است، یک غذا، یا یک مفهوم علمی. بنابراین بردار باید تمام احتمالات آینده را باز نگه دارد (آنتروپی بالا).

- **در مدل BERT (دوجهتی):** هنگام پردازش love، مکانیزم توجه می‌تواند به کلمه AI که در آینده (سمت راست) قرار دارد، نگاه کند. در نتیجه بردار Embedding کلمه love با اطلاعات کلمه AI ترکیب می‌شود. معنای کلمه در اینجا دیگر صرفاً «عشق ورزیدن» نیست، بلکه به معنای «علاقه به تکنولوژی» تخصصی می‌شود. این ویژگی برای درک معنایی دقیق (مانند تحلیل احساسات یا رفع ابهام کلمات چندمعنایی) حیاتی است.

(ه) فرض کنید خروجی self-attention برای [CLS] (که در بخش قبل محاسبه شده) به‌عنوان embedding نهایی استفاده می‌شود و یک تابع هزینه ساده از نوع Mean Squared Error (MSE) داریم:

$$L = \left\|\text{output}_{[\text{CLS}]} - \text{target}\right\|_{۲}^{۲},$$

که در آن

$$\text{target} = [۰/۵,\ ۰/۵]$$

است. گرادیان تابع هزینه L نسبت به تمام ماتریس‌های projection (یعنی W_Q، W_K و W_V) را با استفاده از chain rule در فرایند backpropagation محاسبه کنید. همچنین توضیح دهید چرا backpropagation روی همه projectionها انجام می‌شود و این کار چگونه به به‌روزرسانی وزن‌ها در آموزش مدل کمک می‌کند.

ابتدا مشتق تابع هزینه (L) را نسبت به خروجی نهایی (Z) محاسبه می‌کنیم:

$$L = \|Z - T\|^{۲} \Rightarrow \frac{\partial L}{\partial Z} = ۲(Z - T)$$

با جایگذاری مقادیر عددی بدست آمده از بخش قبل ($Z = [۱/۰, ۰/۴۶]$) و هدف ($T = [۰/۵, ۰/۵]$):

$$\delta_{out} = ۲([۱/۰, ۰/۴۶] - [۰/۵, ۰/۵]) = [۱/۰, -۰/۰۸]$$

طبق قانون زنجیره‌ای:

$$\frac{\partial L}{\partial W_V} = \frac{\partial L}{\partial Z} \cdot \frac{\partial Z}{\partial W_V}$$

می‌دانیم $Z = CW_V$ که در آن C مجموع ورودی‌های وزن‌دار شده است ($C = \sum \alpha_i x_i \approx [۰/۵۴, ۰/۴۶]$). بنابراین:

$$\frac{\partial L}{\partial W_V} = C^{\top}\delta_{out} = \begin{pmatrix} ۰/۵۴ \\ ۰/۴۶ \end{pmatrix} \begin{pmatrix} ۱/۰ & -۰/۰۸ \end{pmatrix}$$

$$\frac{\partial L}{\partial W_V} \approx \begin{pmatrix} ۰/۵۴ & -۰/۰۴ \\ ۰/۴۶ & -۰/۰۳ \end{pmatrix}$$

برای رسیدن به W_Q و W_K باید خطا را از لایه Softmax عبور دهیم.

$$\frac{\partial L}{\partial \alpha_j} = \delta_{out} \cdot v_j^{\top}$$

با ضرب داخلی δ_{out} در هر بردار v_j:

$$\begin{aligned}
\delta_{\alpha_۱} &= ۱/۰(۱) + (-۰/۰۸)(۰) = ۱/۰ \\
\delta_{\alpha_۲} &= ۱/۰(۱) + (-۰/۰۸)(۱) = ۰/۹۲ \\
\delta_{\alpha_۳} &= ۱/۰(۲) + (-۰/۰۸)(۱) = ۱/۹۲ \\
\delta_{\alpha_۴} &= ۱/۰(۰) + (-۰/۰۸)(۰) = ۰ \\
\delta_{\alpha_۵} &= ۱/۰(-۱) + (-۰/۰۸)(۰) = -۱/۰
\end{aligned}$$

$$\delta_{\alpha} = [۱/۰, ۰/۹۲, ۱/۹۲, ۰, -۱/۰]$$

فرمول مشتق سافت‌مکس: $\frac{\partial L}{\partial S_j} = \alpha_j(\delta_{\alpha_j} - \bar{\delta})$ که در آن $\bar{\delta} = \sum \alpha_k \delta_{\alpha_k}$.

$$\bar{\delta} = (۰/۳۱)(۱) + (۰/۱۵)(۰/۹۲) + (۰/۳۱)(۱/۹۲) + ۰ + (۰/۰۸)(-۱) \approx ۰/۹۶۳$$

حال برای هر درایه:

$$\begin{aligned}
\delta_{S_۱} &= ۰/۳۱(۱/۰ - ۰/۹۶۳) = ۰/۰۱۱ \\
\delta_{S_۲} &= ۰/۱۵(۰/۹۲ - ۰/۹۶۳) = -۰/۰۰۶ \\
\delta_{S_۳} &= ۰/۳۱(۱/۹۲ - ۰/۹۶۳) = ۰/۲۹۷ \\
\delta_{S_۴} &= ۰/۱۵(۰ - ۰/۹۶۳) = -۰/۱۴۴ \\
\delta_{S_۵} &= ۰/۰۸(-۱/۰ - ۰/۹۶۳) = -۰/۱۵۷
\end{aligned}$$

مسیر خطا برای ماتریس وزن کوئری: $S \to q_۱ \to W_Q$.

$$\frac{\partial S_j}{\partial q_۱} = \frac{k_j}{\sqrt{d_k}} \implies \delta_{q_۱} = \sum_j \delta_{S_j} \frac{k_j}{\sqrt{۲}}$$

با جمع زدن $\delta_{S_j} k_j$ و تقسیم بر ۱/۴۱۴:

$$\sum \delta_{S_j} k_j = [۰/۴۶۵, ۰/۷۵۶]$$

$$\delta_{q_۱} = ۰/۷۰۷ \times [۰/۴۶۵, ۰/۷۵۶] = [۰/۳۲۹, ۰/۵۳۴]$$

چون $q_۱ = x_۱ W_Q$ و فقط توکن اول Query تولید کرده:

$$\frac{\partial L}{\partial W_Q} = x_۱^\top \delta_{q_۱} = \begin{pmatrix} ۱ \\ ۰ \end{pmatrix} [۰/۳۲۹, ۰/۵۳۴] = \begin{pmatrix} ۰/۳۲۹ & ۰/۵۳۴ \\ ۰ & ۰ \end{pmatrix}$$

مسیر خطا برای ماتریس وزن کلید: $S \to K \to W_K$.

$$\delta_{k_j} = \delta_{S_j} \frac{q_۱}{\sqrt{۲}}$$

چون $q_۱ = [۱, ۰]$، گرادیان‌ها فقط مولفه افقی دارند (ضرب در ۰/۷۰۷):

$$\delta_k \approx \begin{pmatrix} ۰/۰۰۸ & ۰ \\ -۰/۰۰۴ & ۰ \\ ۰/۲۱۰ & ۰ \\ -۰/۱۰۲ & ۰ \\ -۰/۱۱۱ & ۰ \end{pmatrix}$$

$$\frac{\partial L}{\partial W_K} = \sum_j x_j^\top \delta_{k_j}$$

با محاسبه $x_j^\top \times \delta_{k_j}$ برای هر ۵ توکن و جمع آنها:

$$\frac{\partial L}{\partial W_K} = \begin{pmatrix} ۰/۳۲۹ & ۰ \\ ۰/۲۰۶ & ۰ \end{pmatrix}$$

فرآیند Backpropagation چگونه مدل را بهبود می‌دهد؟

- گرادیان‌ها به ما می‌گویند که هر وزن چقدر در ایجاد خطای نهایی مسئول بوده است.
- **اصلاح مسیر توجه (W_Q, W_K):** گرادیان‌های این ماتریس‌ها، الگوی توجه (Attention-Matrix) را تغییر می‌دهند تا مدل یاد بگیرد در آینده به کلمات مرتبط‌تری (مثل رابطه love و AI) توجه کند.

- **اصلاح استخراج ویژگی (W_V):** گرادیان این ماتریس، بردار ویژگی‌ای که از هر کلمه منتقل می‌شود را تغییر می‌دهد تا اطلاعات مفیدتری برای حل مسئله نهایی استخراج شود.

در نهایت، وزن‌ها با فرمول $W \leftarrow W - \eta \nabla L$ آپدیت می‌شوند تا خطا در دورهای بعدی کاهش یابد.

۴. (۲۰ نمره) در یک لایه ترنسفورمر، برای هر ماتریس خطی $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ یک اصلاح low-rank اضافه می‌کنیم. پارامتردهی رایج LoRA به‌شکل زیر است:

$$A \in \mathbb{R}^{r \times d_{\text{in}}}, \qquad B \in \mathbb{R}^{d_{\text{out}} \times r},$$

$$\Delta W = BA, \qquad W = W_{\cdot} + \Delta W$$

(آ) فرمول کلی تعداد پارامترهای قابل‌آموزش LoRA را برای یک ماتریس $W_{\cdot}$ برحسب $d_{\text{in}}, d_{\text{out}}, r$ به‌دست آورید.

برای یک ماتریس وزن W با ابعاد $d_{out} \times d_{in}$، ما دو ماتریس کم‌رتبه A و B را آموزش می‌دهیم. تعداد پارامترهای این دو ماتریس عبارتند از:

- ماتریس A با ابعاد $r \times d_{in}$: دارای $r \cdot d_{in}$ پارامتر.
- ماتریس B با ابعاد $d_{out} \times r$: دارای $d_{out} \cdot r$ پارامتر.

بنابراین تعداد کل پارامترهای قابل آموزش برابر است با:

$$N_{param} = r \cdot d_{in} + d_{out} \cdot r = r(d_{in} + d_{out})$$

(ب) فرض کنید یک مدل decoder-only با L لایه دارید و در هر لایه attention چهار پروجکشن W_q، W_v، W_k و W_o وجود دارد که همگی اندازه $d \times d$ دارند. اگر LoRA فقط روی Q و V اعمال شود، تعداد پارامترهای قابل‌آموزش کل را برحسب L, d, r محاسبه کنید.

۱. **پارامترهای یک ماتریس:** $r(d+d) = ۲dr$

۲. **تعداد ماتریس‌ها در هر لایه:** ۲ ماتریس (W_q و W_v).

۳. **پارامترهای هر لایه:** $۲ \times ۲dr = ۴dr$

۴. **کل پارامترها برای L لایه:**

$$N_{total} = L \times ۴dr = ۴Ldr$$

(ج) همین محاسبه را برای حالتی انجام دهید که LoRA روی هر چهار ماتریس Q, K, V, O اعمال می‌شود.

۱. **پارامترهای یک ماتریس:** $r(d+d) = ۲dr$

۲. **تعداد ماتریس‌ها در هر لایه:** ۴ ماتریس (W_q و W_v و W_k و W_o).

۳. **پارامترهای هر لایه:** $۴ \times ۲dr = ۸dr$

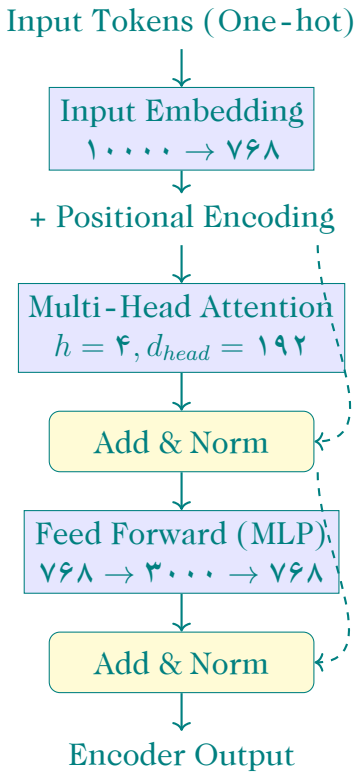
۴. **کل پارامترها برای L لایه:**

$$N_{total} = L \times ۸dr = ۸Ldr$$

۵. (۲۰ نمره) مساله embed کردن متن را در نظر بگیرید. فرض کنید تعداد واژگان دیکشنری برابر ۱۰۰۰۰ باشد و تعداد توکن‌های متن ورودی حداکثر ۵۰۰ در نظر گرفته شده باشد و واژه‌ها به صورت one-hot به عنوان ورودی به شبکه داده شوند.

اگر از معماری ترنسفورمر استفاده شود، فرض کنید که بعد embedding هر لایه self-attention برابر ۷۶۸، بعد مخفی یا همان تعداد گره‌های لایه مخفی MLP تک لایه مخفی برابر ۳۰۰۰ و تعداد head لایه self-attention برابر ۴ و بعد متناظر با هر head برابر ۱۹۲ باشد. فرض کنید از positional embedding با استفاده از توابع سینوسی و کسینوسی ایجاد شده است. شمای کلی شبکه‌ی embedding و یک بلوک انکدر را رسم کنید و سپس تعداد پارامترهای آن را به دست آورید.

در تصویر زیر ساختار یک بلوک انکودر به همراه لایه امبدینگ ورودی ترسیم شده است:



پارامترهای شبکه به تفکیک لایه‌ها محاسبه می‌شوند. فرض بر این است که لایه‌های خطی دارای بایاس (Bias) هستند.

۱. **لایه Embedding**

$$V \times d_{model} = ۱۰,۰۰۰ \times ۷۶۸ = ۷,۶۸۰,۰۰۰$$

۲. **لایه Multi-Head Attention** شامل ماتریس‌های W_Q, W_K, W_V و W_O.

- وزن‌ها: $۴ \times (۷۶۸ \times ۷۶۸) = ۲,۳۵۹,۲۹۶$
- بایاس‌ها: $۴ \times ۷۶۸ = ۳,۰۷۲$
- **مجموع:** $۲,۳۶۲,۳۶۸$

۳. **لایه MLP** شامل دو لایه خطی (ورودی به مخفی و مخفی به خروجی).

- وزن‌ها: $(۷۶۸ \times ۳۰۰۰) + (۳۰۰۰ \times ۷۶۸) = ۴,۶۰۸,۰۰۰$
- بایاس‌ها: $۳۰۰۰ + ۷۶۸ = ۳,۷۶۸$

- **مجموع:** ۴,۶۱۱,۷۶۸

۴. لایه Normalization Layer دو لایه نرمال‌سازی با پارامترهای یادگیرنده γ و β.

$$۲ \times (۲ \times ۷۶۸) = ۳,۰۷۲$$

مجموع کل پارامترهای شبکه برابر است با:

$$۷,۶۸۰,۰۰۰ + ۲,۳۶۲,۳۶۸ + ۴,۶۱۱,۷۶۸ + ۳,۰۷۲ = ۱۴,۶۵۷,۲۰۸$$